

Sesión 7: Árboles Binarios de Búsqueda (ABB)

Propiedades Fundamentales, Algoritmos de Inserción Condicionada, Estrategias de Búsqueda de
Llaves y Construcción en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

Agenda del Día (1/2)

- ☰ Propiedad de Ordenación de un ABB
- ↓ Regla Matemática: Menores a la izquierda, mayores a la derecha
- 🔑 El papel de la Recursividad en las operaciones no lineales
- ➕ Algoritmo de Inserción de nuevas llaves paso a paso



Optimización del Acceso

Cómo transformar la búsqueda lineal $O(n)$ en un eficiente recorrido de árbol acotado por su altura.

Agenda del Día (2/2)



Código C# Completo

Construcción del método de inserción y búsquedas en modo consola sin librerías externas.



Análisis Temporal

Evaluación de la eficiencia en el mejor de los casos frente a la degradación lineal del peor caso.



Responsabilidad

Valor de la unidad: Compromiso con el diseño estructural óptimo y robusto.



La Propiedad de Ordenación del ABB

Estableciendo las reglas restrictivas que permiten búsquedas eficientes

Definición de un Árbol Binario de Búsqueda

Un **Árbol Binario de Búsqueda (ABB)** es un árbol binario estructurado con una propiedad fundamental de ordenación obligatoria para cada uno de sus nodos:

La Regla de Oro Invariable:

- Todos los elementos del **subárbol izquierdo** deben tener un valor **menor** que la clave del nodo padre.
- Todos los elementos del **subárbol derecho** deben tener un valor **mayor o igual** que la clave del nodo padre.

Visualización Gráfica de un ABB Coherente

Observemos cómo se distribuyen los valores respetando la regla en cada ramificación:



El 8 es menor que 15 (va a la izquierda). El 24 es mayor que 15 (va a la derecha). Esta propiedad se mantiene en cascada infinita.



Algoritmo Lógico de Inserción

Cómo ubicar un nuevo elemento en el lugar exacto sin violar las restricciones

Mecánica de la Inserción Recurrente

Para insertar una nueva llave "X" en el árbol, el algoritmo sigue estos pasos lógicos:

Paso	Condición de Verificación	Acción Operativa
1	¿El nodo actual es nulo?	Encontramos la posición libre. Se crea y coloca el nodo aquí.
2	¿La llave "X" es menor al nodo actual?	Nos desplazamos recursivamente hacia el Subárbol Izquierdo .
3	¿La llave "X" es mayor o igual?	Nos desplazamos recursivamente hacia el Subárbol Derecho .



Construcción del ABB en C#

Traducción de los diagramas conceptuales a código funcional de .NET

Estructura Base de la Clase Controladora

Partimos del modelo definido en la sesión previa, agregando el punto de acceso:

```
public class ArbolBinarioBusqueda {  
    private NodoArbol raiz;  
  
    public ArbolBinarioBusqueda() {  
        raiz = null;  
    }  
  
    // Punto de entrada público para el cliente de la clase  
    public void Insertar(int valor) {  
        raiz = InsertarRecursivo(raiz, valor);  
    }  
}
```


Análisis de la Encapsulación en la Inserción

El método público `Insertar(int valor)` oculta la complejidad del manejo de punteros al usuario final del objeto.

****Flujo de ocultamiento:****

- El programador externo solo invoca `arbol.Insertar(10);`.
- No necesita saber qué es la propiedad raíz ni cómo se manipulan los enlaces lógicos de memoria interna.

🛡️ Seguridad de la Raíz:

Al mantener la raíz como un atributo privado, evitamos que código externo rompa de forma accidental los enlaces de los subárboles.

Código: El Corazón del Algoritmo de Inserción

```
private NodoArbol InsertarRecursoivo(NodoArbol actual, int valor) {  
    // Caso Base: Si el espacio está libre, instanciamos el nuevo nodo  
    if (actual == null) {  
        return new NodoArbol(valor);  
    }  
  
    // Decisiones lógicas de bifurcación jerárquica  
    if (valor < actual.Dato) {  
        actual.Izquierdo = InsertarRecursoivo(actual.Izquierdo, valor);  
    } else {  
        actual.Derecho = InsertarRecursoivo(actual.Derecho, valor);  
    }  
  
    return actual; // Retorna la referencia del nodo modificado  
}
```


Desglose de Código: El Caso Base

Analicemos las primeras líneas del método recursivo:

```
if (actual == null) { return new NodoArbol(valor); }
```

- ****¿Cuándo ocurre?**** Cuando el puntero de navegación llega a un espacio vacío (la raíz de un árbol vacío o la rama libre de una hoja).
- ****Mecánica:**** Llama al constructor de `NodoArbol` y le devuelve esa nueva dirección de memoria al marco de ejecución padre para que este reemplace su antiguo enlace ``null``.

Desglose de Código: Decisiones de Camino

Evaluemos la bifurcación condicional:

```
if (valor < actual.Dato) { actual.Izquierdo = InsertarRecursoivo(...); }
```

Si la llave nueva es menor que la del nodo actual, delegamos la tarea al **subárbol izquierdo**. Al hacer la asignación `actual.Izquierdo = ...`, estamos asegurando que cuando la recursividad encuentre el caso base y cree el nodo, el enlace quede guardado de forma permanente.



Algoritmo Lógico de Búsqueda

El poder de descartar mitades enteras de datos con un solo criterio condicional

¿Cómo funciona la consulta de llaves?

La búsqueda aprovecha el ordenamiento físico del ABB para evitar recorrer toda la colección de elementos:

¿Es idéntico?

Si el valor buscado coincide con la llave del nodo actual, la búsqueda concluye con éxito instantáneo.

¿Es menor?

Ignoramos por completo todos los elementos del lado derecho y viajamos al subárbol izquierdo.

¿Es mayor?

Ignoramos por completo el subárbol izquierdo y enfocamos los recursos en el lado derecho.

Código: Métodos de Búsqueda de Claves

Añadimos la rutina de búsqueda lógica a nuestra clase en C#:

```
public bool Buscar(int valor) {  
    return BuscarRecursoivo(raiz, valor);  
}  
  
private bool BuscarRecursoivo(NodoArbol actual, int valor) {  
    if (actual == null) return false; // No se encontró el valor  
    if (valor == actual.Dato) return true; // ¡Encontrado!  
  
    return valor < actual.Dato  
        ? BuscarRecursoivo(actual.Izquierdo, valor)  
        : BuscarRecursoivo(actual.Derecho, valor);  
}
```


Desglose de Búsqueda: Casos de Salida

El método `BuscarRecursivo` posee dos condiciones de salida deterministas:

- ****Salida por Ausencia:**** `if (actual == null) return false;` Significa que navegamos por las ramas hasta un extremo vacío sin encontrar coincidencia del dato.
- ****Salida por Éxito:**** `if (valor == actual.Dato) return true;` Detiene de inmediato el avance recursivo devolviendo una señal de éxito hacia arriba.

Desglose de Búsqueda: Ramificación Limpia

Analicemos la instrucción de retorno condicional simplificado:

```
return valor < actual.Dato ? Buscar(...Izquierdo...) : Buscar(...Derecho...);
```

Haciendo uso del operador ternario de C#, decidimos el camino de forma compacta. Si la condición se evalúa como verdadera, se ejecuta el viaje al hijo izquierdo; si es falsa, se traslada al hijo derecho.

Análisis de Complejidad Asintótica

Midiendo el rendimiento científico del ABB mediante la notación formal Big-O

El Caso Ideal: Árbol Balanceado

Cuando el árbol se encuentra perfectamente equilibrado, la altura máxima está acotada por $\log_2 n$.

Complejidad en el Mejor Caso = $O(\log n)$

En cada paso descartamos la mitad de los datos del sistema. Buscar un elemento entre un millón requerirá **únicamente un aproximado de 20 comparaciones lógicas**.

El Peor Caso: Árbol Degenerado

Si los datos ingresan ordenados de forma secuencial (por ejemplo: 1, 2, 3, 4, 5), los elementos se colocarán todos del mismo lado.

Complejidad en el Peor Caso = $O(n)$

El árbol se deforma convirtiéndose físicamente en una **Lista Enlazada**. Perderíamos de inmediato toda ventaja jerárquica (problema que solucionaremos al estudiar Árboles AVL en la sesión 9).

Orquestación Práctica en el Método Main

Simulación completa de inserción secuencial y verificación de búsquedas en consola

Código: Flujo de Ejecución de Pruebas

Unimos todos los componentes en el punto de entrada principal del software:

```
public static void Main(string[] args) {  
    ArbolBinarioBusqueda miArbol = new ArbolBinarioBusqueda();  
  
    // Construyendo la jerarquía  
    miArbol.Insertar(15);  
    miArbol.Insertar(8);  
    miArbol.Insertar(24);  
  
    // Evaluando consultas lógicas  
    Console.WriteLine("¿Existe el 24?: " + miArbol.Buscar(24)); // True  
    Console.WriteLine("¿Existe el 99?: " + miArbol.Buscar(99)); // False  
}
```


Trazabilidad Interna de las Referencias

Al concluir las instrucciones del método Main, las variables apuntan en memoria de esta forma:

```
[miArbol] → raiz (Dirección: 0x00A1)
          |—— Dato: 15
          |—— Izquierdo → Nodo (0x00B2) [Dato: 8 | Izq: null | Der: null]
          |—— Derecho → Nodo (0x00C3) [Dato: 24 | Izq: null | Der: null]
```

Errores Comunes y Estrategias de Depuración

Cómo evitar bucles infinitos por mal manejo de asignaciones recursivas

Error Clásico: Omitir el Retorno del Nodo

Un error recurrente en los laboratorios de IPC 2 es olvidar capturar el valor de retorno en la llamada recursiva:

```
// CÓDIGO ERRÓNEO: El árbol nunca crecerá porque el padre no guarda el enlace  
InsertarRecursivo(actual.Izquierdo, valor);
```

✖ ****Consecuencia:**** El método crea el nodo en memoria, pero al salir del marco de ejecución, el puntero del padre sigue apuntando a `null`, perdiendo el nodo de forma inmediata en el recolector de basura (*Garbage Collector*).

Valor de la Unidad: Responsabilidad



"No se puede escapar de la responsabilidad del mañana evadiéndola hoy."

— Abraham Lincoln

****Aplicación Práctica:**** Es nuestra responsabilidad como desarrolladores probar los algoritmos con casos límite para asegurar que la estructura responda correctamente y con un alto desempeño en producción.

Conclusiones de la Sesión

Regla Estricta

La propiedad de ordenación (menores a la izquierda, mayores a la derecha) transforma un árbol genérico en una potente herramienta de indexación.

Poder Logarítmico

La búsqueda en un ABB óptimo se ejecuta en tiempo $O(\log n)$, lo cual permite realizar búsquedas en colecciones gigantescas de datos de forma instantánea.

Manejo de Enlaces

La asignación explícita mediante funciones recursivas es la manera correcta y segura de enlazar nuevos nodos en memoria dinámica de C#.

Próximo Paso: Sesión 8

Recorridos y Eliminación en Árboles Binarios

En la siguiente sesión aprenderemos a extraer la información ordenada mediante **Recorridos Inorden, Preorden y Postorden**, así como la compleja lógica requerida para **eliminar nodos** según sus tres casos operativos.

¿Dudas de la Construcción de un ABB?

Es momento de configurar sus entornos.

 Foros en UEDI  Soporte: 3021894750101@ingenieria.usac.edu.gt